Amazon Elastic Container Registry

Explore Amazon Elastic Container Registry (ECR) to understand how to securely store and manage Docker images on AWS. Learn key processes including authentication, image tagging, pushing images, and lifecycle management. Discover how IAM access, encryption, and vulnerability scanning enhance security. Gain insights into troubleshooting common issues to maintain smooth container workflows within AWS environments.

Working with Docker makes one realize how valuable container images are in enabling the packaging of an application and its dependencies into a portable unit. Whether we're building a microservice architecture or deploying serverless functions that require container runtimes, Docker images are central to modern application delivery.

Organizations deploying multiple containers require a registry to store, maintain, and distribute different images. A container registry is crucial in the Docker ecosystem to provide a reliable and scalable infrastructure for sharing and collaborating on container images across different environments.

We can use the Amazon Elastic Container Registry (ECR) for container management. It enables reliable and efficient container deployment, facilitates version control, and ensures consistency in application delivery.

Amazon Elastic Container Registry

Amazon Elastic Container Registry (ECR) is an AWS-managed Docker container registry service. It provides a secure and scalable repository for storing and managing Docker images. It offers both public and private registries and allows us to store multiple repositories in these registries.

Ask

Amazon ECR supports two types of registries: private and public, each suited for different collaboration and distribution models:

- ECR is primarily designed as a private registry, meaning that it's intended for use within organizations or teams. We can push our Docker images to ECR and control access permissions to ensure that only authorized users can pull or modify these images. These private registry repositories are available only to authorized IAM entities.
- ECR also offers public registries, which are container registries publicly accessible to anyone on the internet. It hosts a wide range of open-source and community-contributed Docker images, making them readily available for developers to pull and use in their projects.

Types of registries

Ask

Working with ECR repositories

When adding an image to an Amazon ECR repository, there are a few core steps that we must follow. This process ensures that our Docker image is properly built, securely stored, and made available to our compute services. The high-level workflow typically includes the following phases:

1. **Authentication:** We start by authenticating the Docker client with the Amazon ECR using the AWS CLI. This step is crucial because ECR requires authentication before it allows any image operations.
2. **Build the Docker image:** Once authenticated, we use the `docker build` command to package our application and dependencies into an image.
3. **Tag the image:** Before pushing, the image must be tagged with the full URI of the ECR repository, so Docker knows where to upload it.

> **Note:** Tagging Docker images in Amazon ECR is crucial for managing versioned deployments across environments. Strategies like semantic versioning, commit hashes, or environment-specific tags improve clarity, traceability, and consistency. This practice prevents accidental overwrites, supports reliable CI/CD workflows, and ensures deployment of the correct image version.

4. **Push the image:** Finally, the image is pushed to ECR using the `docker push` command.

Here are the corresponding commands for these phases:

Step 1

```
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin <aws_account_id>.dkr.ec
```

Ask

Log in to ECR

Once the image is pushed to Amazon ECR, it can be pulled and used just like images from Docker Hub. Any container orchestration service, such as Amazon ECS, EKS, or even a local Docker environment, can reference the image using its ECR URI. As long as the necessary permissions and authentication are in place, these images can seamlessly integrate into containerized workflows, deployments, or CI/CD pipelines.

Clean up using lifecycle policies

As container repositories grow, managing stale or redundant images becomes crucial for maintaining efficiency and controlling storage costs. Amazon ECR provides built-in lifecycle policies to automate image clean up.

Lifecycle policies allow us to define rules to automatically delete unused or outdated images from our repository. This not only reduces clutter but also ensures our CI/CD workflows operate on up-to-date, actively maintained builds.

Here's an example of a JSON-based lifecycle policy that deletes images created over 30 days ago:

index.js C++

```
{
    "rules": [
        {
            "rulePriority": 1,
            "description": "Expire images older than 30 days",
            "selection": {
```

Ask

```
                "tagStatus": "tagged",
                "tagPatternList": ["prod*"],
                "countType": "sinceImagePushed",
                "countUnit": "days",
                "countNumber": 30
            },
```

Lifecycle policy

We can apply the policy using the following AWS CLI command:

main.sh Shell

```
aws ecr put-lifecycle-policy \
  --repository-name my-app \
  --lifecycle-policy-text file://policy.json
```

Apply lifecycle policy using CLI

Securing repositories

Securing container images is vital for maintaining trust in the application pipeline. Amazon ECR provides multiple layers of protection and security integrations to ensure images are accessible only to authorized entities and free from known vulnerabilities before deployment.

Ask

IAM based access control

Amazon ECR integrates with IAM to define fine-grained access control for users, roles, and services interacting with repositories. Permissions can be scoped to allow push-only, pull-only, or full repository access. For example, the following policy allows a user to pull images from a specific repository:

index.js Javascript (babel-node)

```
{
  "Effect": "Allow",
  "Action": [
    "ecr:GetDownloadUrlForLayer",
    "ecr:BatchGetImage"
  ],
  "Resource": "arn:aws:ecr:us-east-1:<account_id>:repository/my-app"
}
```

Permissions to allow image pulls

> IAM policies should always follow the principle of least privilege, only granting the permissions necessary for the intended task.

Encryption with AWS KMS

By default, Amazon ECR encrypts images at rest using AWS managed keys. We can opt for customer managed keys (CMKs) to enforce more stringent compliance or cross-account controls. We can specify the CMK to use during reposito creation as follows:

Ask

main.sh Shell

```shell
aws ecr create-repository \
  --repository-name secure-repo \
  --encryption-configuration encryptionType=KMS,kmsKey=<key_arn>
```

Encryption using CMKs

This ensures that container artifacts meet internal and regulatory standards for data protection.

Image scanning for vulnerabilities

To help maintain security and compliance, Amazon ECR provides built-in image scanning capabilities that identify known vulnerabilities in container images. This ensures potential risks are identified early in the development lifecycle. It supports two types of image scanning:

- **Basic scanning:** It compares container images against the Common Vulnerabilities and Exposures (CVE) database using Amazon Inspector during on-push scans.
- **Enhanced scanning:** It provides deeper inspection and continuous scanning using advanced tooling.

We can enable basic scanning on an existing ECR repository by using the following AWS CLI command:

Ask

main.sh Shell

```shell
aws ecr put-image-scanning-configuration \
  --repository-name my-app \
  --image-scanning-configuration scanOnPush=true
```

Enable basic scanning

After images are pushed, we can retrieve scan findings by executing the following AWS CLI command:

main.cpp C++

```cpp
aws ecr describe-image-scan-findings --repository-name my-app --image-
```

Ask

Get findings for the built image

Each finding includes a vulnerability description along with its severity level, such as `LOW`, `MEDIUM`, `HIGH`, or `CRITICAL`.

Run image scan

Integrating image scans into CI/CD pipelines ensures that only clean, validated images progress to staging or production.

By combining IAM, scanning, and encryption, Amazon ECR provides a secure foundation for containerized workflows on AWS. These measures ensure that access is restricted to authorized users and services, vulnerabilities are identified before deployment, and sensitive data remains protected.

Troubleshooting issues in ECR

While Amazon ECR streamlines container image storage and management, developers may occasionally encounter operational issues that disrupt workflows. These problems are often simple to resolve but can cause deployment delays if not addressed proactively.

Here are a few important considerations to keep in mind when troubleshooting ECR-related issues:

Ask

- **Permission errors:** These often occur when IAM roles or users lack the necessary access to push or pull from a repository. To resolve these issues,

we need to review and update the relevant IAM policies as needed.

- **Token expiration:** ECR requires authentication, and the authentication tokens expire every 12 hours. Always regenerate tokens using the AWS CLI before attempting operations, especially in manual sessions or scripts.
- **Pipeline failures:** CI/CD jobs may fail if they use outdated login credentials. Ensure the pipeline includes a step to fetch a fresh token during each run.
- **Tag mismatches:** Referencing mutable tags like `latest` can cause inconsistencies across environments. Use immutable tags (e.g., `v1.0.2`) or SHA digests for consistency.
- **Overwrites and confusion:** Reusing tags without updating references can silently overwrite images. A clear tagging strategy using semantic versions or built metadata helps maintain traceability.

Conclusion

Amazon Elastic Container Registry (ECR) provides a secure, scalable, and fully managed solution for storing and managing Docker container images within the AWS ecosystem. With support for image versioning, access control through IAM, vulnerability scanning, encryption, and lifecycle automation, ECR enables developers to maintain reliable and production-ready container workflows. Whether integrated into CI/CD pipelines or used across ECS, EKS, or local development environments, ECR offers the flexibility, security, and performance needed to build and deploy containerized applications at scale.

Ask